

ASTROVA

AI-Powered Multilingual Indian Heritage Exploration Platform

Smart India Hackathon 2026 Project Report

Prepared from the project source-of-truth, diagram data package,
final report, presentation content and demo plan supplied for Astrova.

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Proposed Solution
6. Scope
7. Technology Stack
8. System Architecture
9. Functional Requirements
10. Non-Functional Requirements
11. System Modules
12. Database Design
13. Data Flow Diagrams
14. Use Case Diagram
15. Class / Component Architecture
16. Activity Diagram
17. User Flow
18. API Architecture
19. Authentication & Security
20. Admin Portal
21. Favorites & User Data Isolation
22. Map & Heritage Discovery
23. Search & Filtering
24. Collections
25. Timeline
26. AI / Chatbot Architecture

27. Testing & Verification

28. Bugs Found & Resolved

29. Current Database Statistics

30. Limitations

31. Future Scope

32. Conclusion

33. References

Appendix A — Demonstration Screens

1. Abstract

Astrova is an AI-powered multilingual Indian cultural heritage exploration platform designed to bring fragmented heritage information into a unified digital experience. The supplied project documentation describes a full-stack web application combining heritage discovery, interactive geographic exploration, historical timelines, curated collections, full-text search, user authentication, personalized favorites, and an administrator content-management portal. The active application is built around Next.js, React, TypeScript, Express.js and Neon PostgreSQL, with Leaflet and OpenStreetMap used for geographic exploration. The chatbot architecture and knowledge base are preserved but the chatbot is currently marked under construction.

The documented data set contains 74 heritage entities, 54 locations, 9 historical periods, 72 media records, 18 sources, 49 heritage relationships, 6 collections, 98 collection items, 12 supported states and 107 chatbot knowledge records. The report below treats the supplied project source-of-truth and diagram data package as the primary evidence and does not silently replace unsupported implementation details with assumptions.

2. Introduction

India's cultural heritage spans monuments, crafts, traditions, festivals, historic places and living cultural practices. The project documentation identifies fragmentation of heritage information, language barriers, limited digital access and lack of contextual exploration as key problems. Astrova addresses these concerns by organizing heritage records with geographic, historical, media and source metadata and exposing them through a modern discovery-oriented interface.

The intended experience follows a discovery journey: a visitor enters Astrova, searches or explores by map, opens a heritage record, reads its story, examines media and location context, discovers related heritage, and continues exploring. Registered users additionally receive personalized favorites, while administrators maintain the content through protected management interfaces.

3. Problem Statement

- Heritage information is distributed across disparate sources and formats.
- Users may face language and accessibility barriers when discovering cultural knowledge.
- Individual heritage records often lack geographic, historical and relational context.
- Lesser-known cultural heritage requires better digital documentation and discoverability.
- A unified platform is needed for discovery, curation, geographic exploration and content administration.

4. Objectives

1. Create a unified digital platform for Indian cultural heritage exploration.
2. Provide interactive map-based discovery using Leaflet and OpenStreetMap.
3. Organize heritage into curated collections and historical periods.
4. Provide full-text search and filtering for discovery.

5. Provide secure user authentication and persistent, per-user favorites.
6. Provide a protected Admin Portal for content CRUD and dashboard statistics.
7. Maintain multilingual AI/chatbot infrastructure for future development.
8. Provide responsive, accessible and polished user experience.
9. Apply layered application security including JWT authentication, API keys and rate limiting.

5. Proposed Solution

Astrova integrates structured heritage data, geographic discovery, historical organization and content management in one web application. Heritage entities are connected to locations, historical periods and sources, enriched with media and related-heritage relationships, and optionally grouped into collections. Authentication adds a user-specific layer for favorites. Administrative workflows provide controlled creation, editing and removal of the underlying content.

The AI/chatbot subsystem is deliberately retained as part of the architecture and database design, but the supplied documentation identifies it as under construction rather than a finished user-facing capability.

6. Scope

Area	Included	Current Status
Heritage	Browse, details, metadata, relationships, media	Implemented
Map	Leaflet/OpenStreetMap, coordinates, markers	Implemented
Search	Full-text search and suggestions	Implemented
Timeline	Historical periods and chronological exploration	Implemented
Collections	Curated collections and collection items	Implemented
Authentication	Register/login, JWT, HttpOnly cookie	Implemented
Favorites	Authenticated user favorites with isolation	Implemented
Admin	Heritage, media, locations, sources, periods, collections, users, dashboard	Implemented
AI/Chatbot	Knowledge architecture and preserved backend/database	Under Construction

7. Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 16.3.2	App Router web application
Frontend	React 19.2.8	UI components and interaction
Frontend	TypeScript	Static typing
Frontend	Tailwind CSS v4	Styling
Frontend	Motion	UI animation
Frontend	Leaflet	Interactive map rendering
Backend	Express 4.21.2	HTTP API server
Backend	TypeScript	Backend type safety
Backend	pg 8.23.0	PostgreSQL access
Backend	jsonwebtoken 9.0.3	JWT authentication
Backend	bcrypt 6.0.0	Password hashing
Database	Neon PostgreSQL	Persistent relational storage
Mapping	OpenStreetMap	Map tile source

8. System Architecture

The documented active architecture follows a layered request path. The browser renders the Next.js application. Requests are routed through the Next.js API proxy, which attaches the server-side API key and forwards authentication cookies. Express middleware performs API-key, authentication, rate-limit and admin checks as appropriate before route handlers and database queries execute.

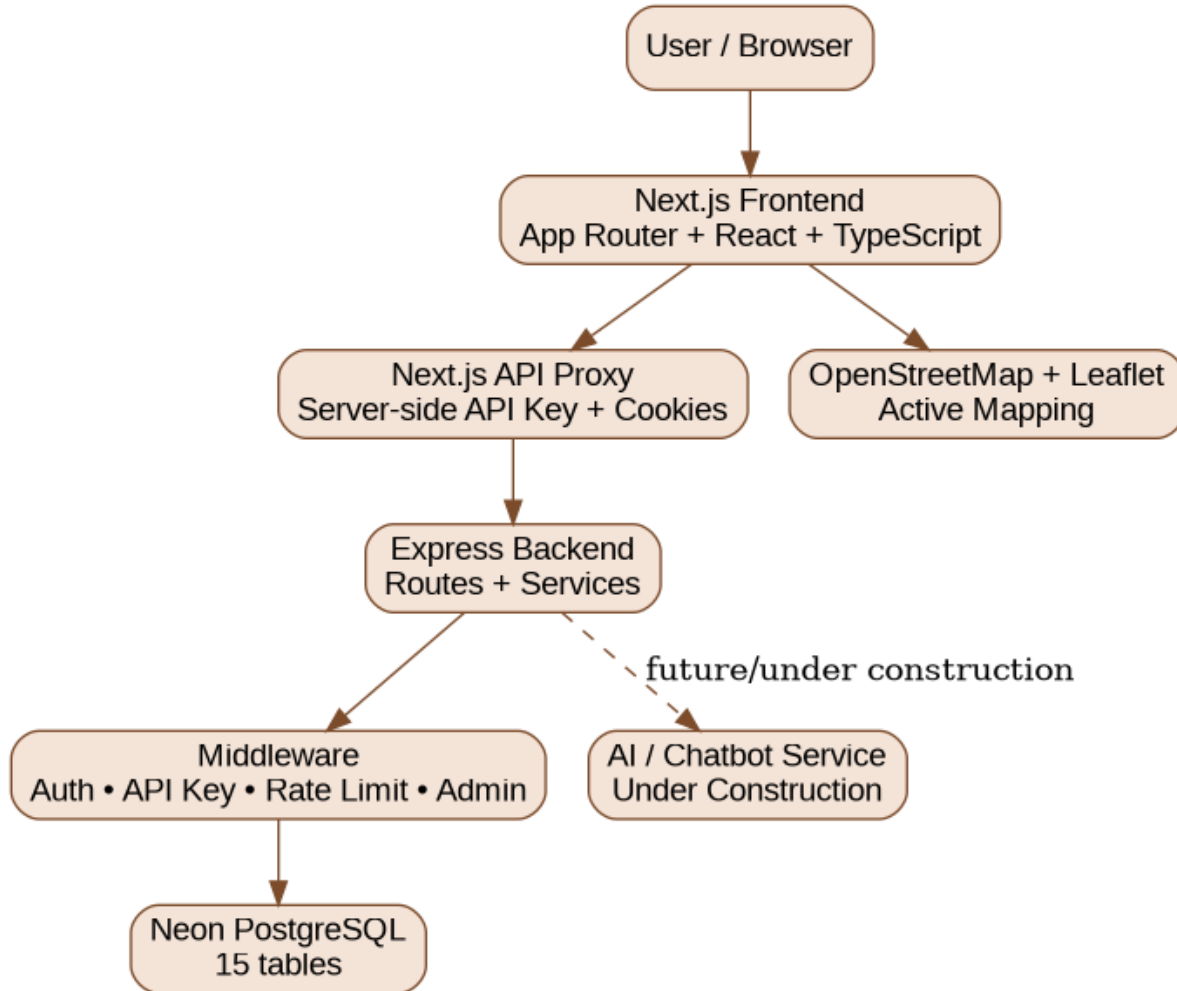


Figure 1. Astrova system architecture derived from the supplied architecture documentation.

Layer	Responsibility
Browser	User interaction, rendering and client state.
Next.js Frontend	Routes, React UI, hooks and feature components.
API Proxy	Server-side API key attachment and cookie forwarding.
Express Backend	HTTP handling, validation, business logic and persistence operations.
Middleware	API key validation, JWT authentication, rate limiting and Admin authentication.
Neon PostgreSQL	Persistent relational data storage.
Leaflet/OpenStreetMap	Geographic visualization and map tiles.

9. Functional Requirements

1. FR-01: Users shall browse heritage records.
2. FR-02: Users shall search heritage records and receive suggestions/results.
3. FR-03: Users shall view detailed heritage information including historical and geographic context.
4. FR-04: Users shall explore heritage through an interactive map.
5. FR-05: Users shall browse historical periods and related heritage.
6. FR-06: Users shall browse collections and collection contents.
7. FR-07: Users shall register and authenticate.
8. FR-08: Authenticated users shall add and remove personal favorites.
9. FR-09: The system shall prevent cross-user favorite visibility and deletion.
10. FR-10: Administrators shall manage heritage, media, locations, sources, periods, collections and users.
11. FR-11: Administrators shall view database-backed dashboard statistics.
12. FR-12: The system shall preserve chatbot infrastructure for future multilingual development.

10. Non-Functional Requirements

- Security: sensitive credentials and secrets must remain server-side and protected.
- Reliability: validation and error handling should prevent invalid database operations.
- Usability: discovery should be understandable through search, map, timeline and collections.
- Responsiveness: core interfaces should work across desktop, tablet and mobile layouts.
- Maintainability: TypeScript, layered routes/services and structured documentation support maintenance.
- Performance: database-backed queries should avoid unnecessary client-side data duplication where practical.
- Accessibility: controls and media should have meaningful labels/alternative text where implemented.

11. System Modules

Module	Description
Heritage Module	Core heritage records, detail pages, metadata, media, location and historical context.
Map Module	Interactive Leaflet/OpenStreetMap exploration using stored coordinates.
Search Module	Full-text heritage discovery and suggestions.
Timeline Module	Chronological browsing across 9 historical periods.
Collections Module	Curated groups of heritage entities using collection_items.
Authentication Module	Registration/login, bcrypt password hashing, JWT and HttpOnly session cookie.
Favorites Module	Authenticated user-specific favorites with backend ownership enforcement.
Admin Module	Protected content management and dashboard statistics.
AI/Chatbot Module	Multilingual knowledge architecture and conversation storage, currently under construction.

12. Database Design

The diagram data package identifies 15 database tables. The central heritage_entities table connects to locations, historical_periods and sources, and is referenced by media, heritage relationships, collection items and user favorites. Additional tables support users, collections, chatbot knowledge, conversation tracking, supported states and analytics.

Table	Purpose	Record Count
heritage_entities	Core heritage data	74
locations	Geographic coordinates and place information	54
historical_periods	Time periods for heritage dating	9
media	Images and videos for heritage entities	72
sources	Reference sources for heritage information	18
heritage_relationships	Connections between heritage entities	49
collections	Curated heritage groupings	6
collection_items	Junction table for collections and heritage	98
supported_states	Indian states supported by the platform	12
users	Registered user accounts	2+
user_favorites	Per-user heritage favorites	varies
chatbot_knowledge	AI chatbot training data	107
conversations	Chatbot conversation sessions	0
conversation_messages	Individual chatbot messages	0

12.1 E-R Diagram

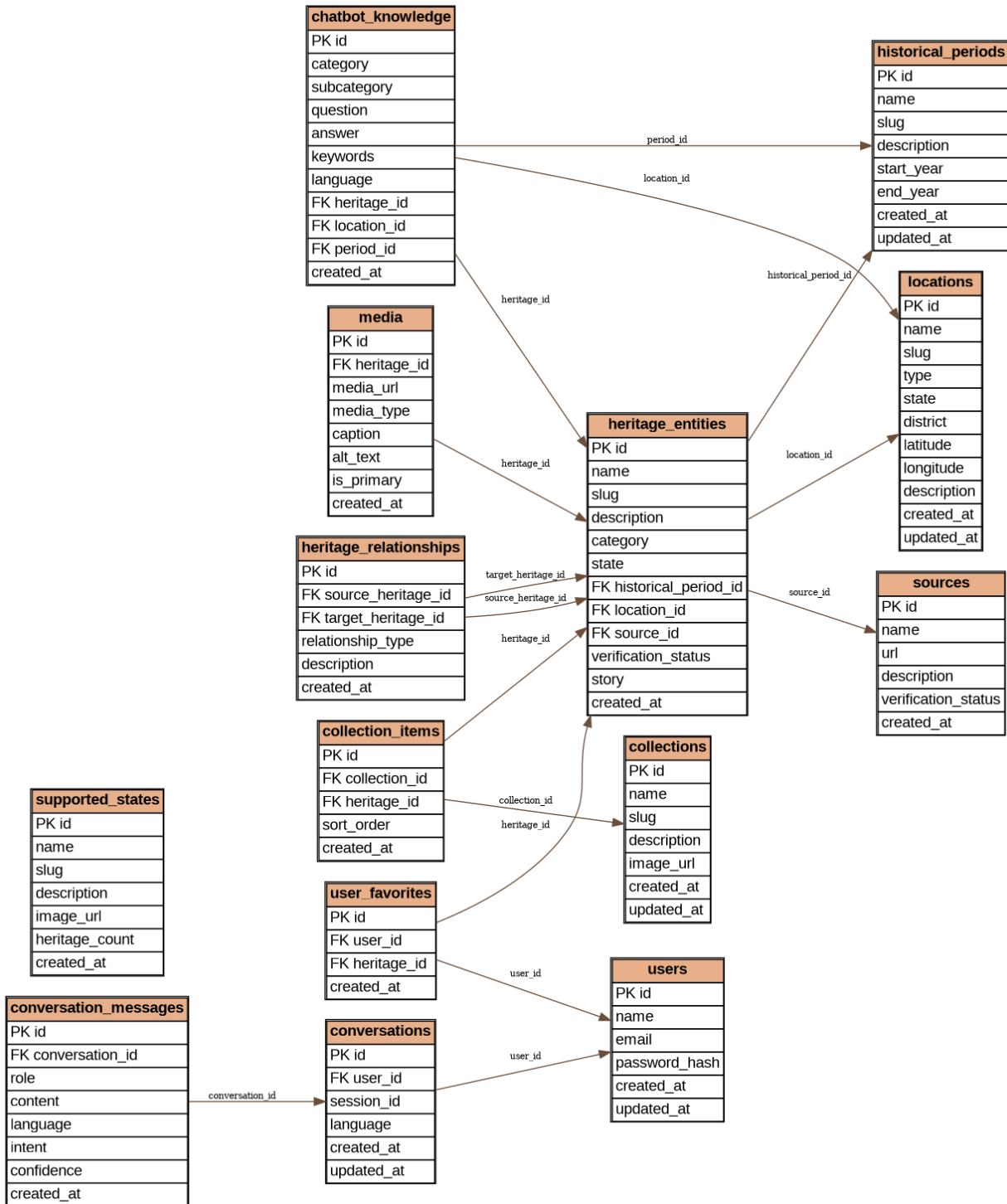


Figure 2. Entity Relationship Diagram generated from the supplied diagram-data package. PK = primary key; FK = foreign key.

The E-R diagram is based on the supplied structured database definitions. The `heritage_relationships` table forms a self-referential many-to-many relationship between heritage entities, while `collection_items` forms the junction between collections and heritage entities. `user_favorites` links users to heritage entities and enforces uniqueness per user and heritage record.

13. Data Flow Diagrams

13.1 DFD Level 0 — Context Diagram

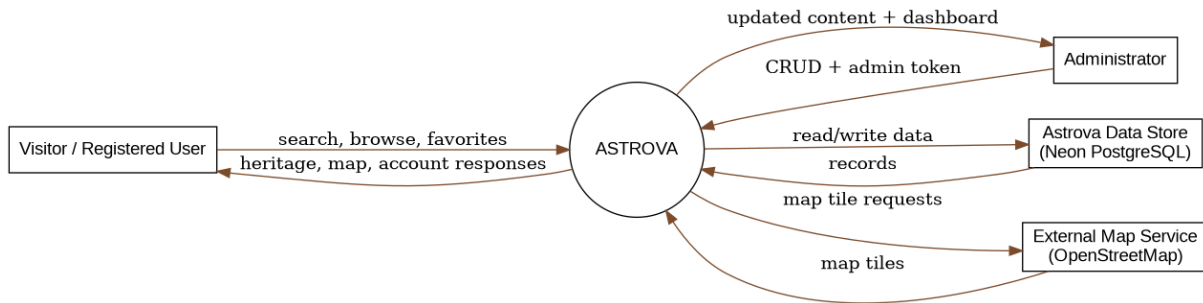


Figure 3. DFD Level 0 — Astrova context.

At context level, Astrova interacts with visitors/registered users, administrators, the persistent PostgreSQL data store and the external OpenStreetMap mapping service.

13.2 DFD Level 1

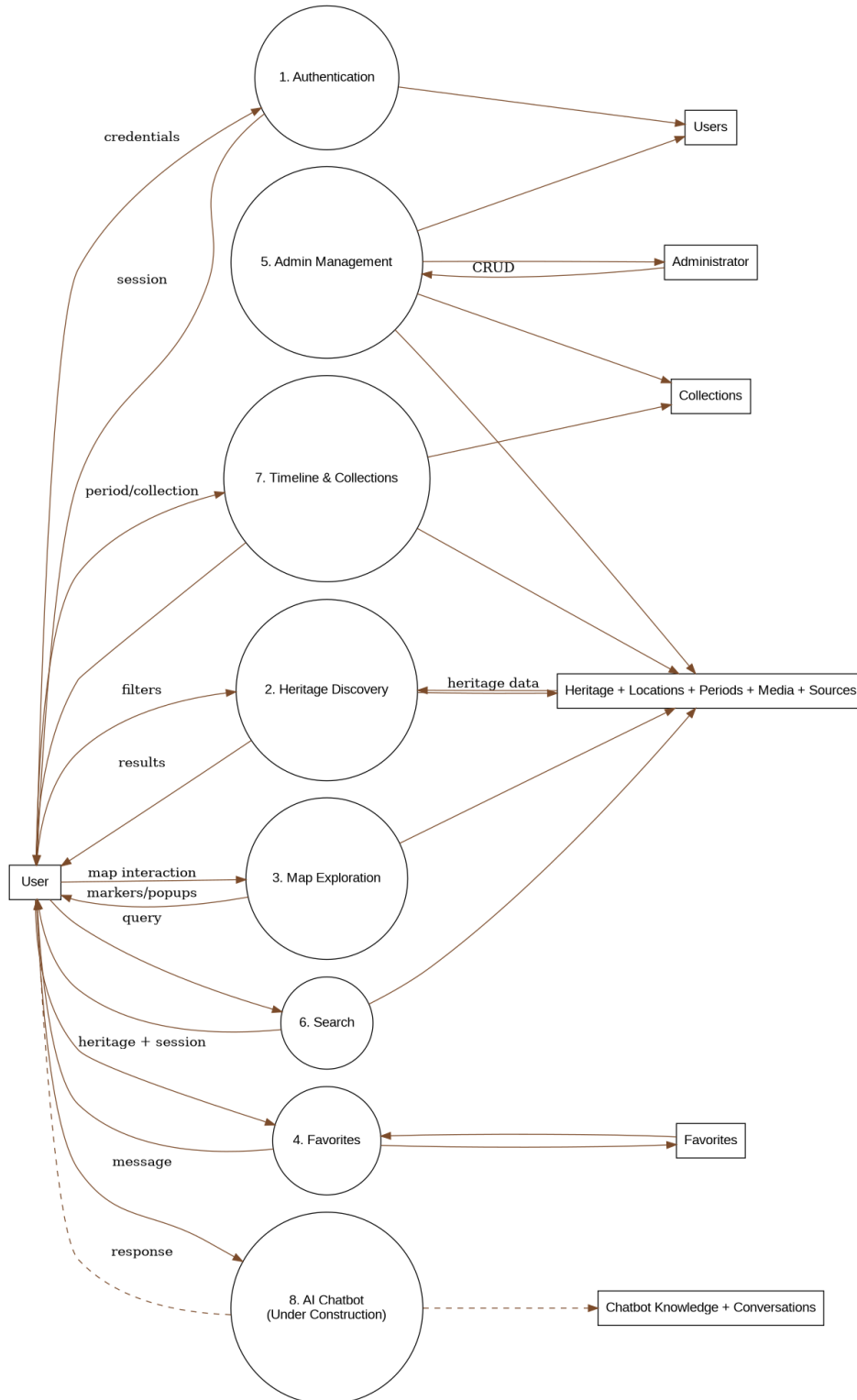


Figure 4. DFD Level 1 — major Astrova processes and data stores.

The Level 1 model decomposes the system into authentication, heritage discovery, map exploration, favorites, administration, search, timeline/collections and the preserved AI chatbot subsystem.

13.3 DFD Level 2 — Representative User/Favorite Flow

The detailed user activity and favorites flow is represented in the activity diagram and user-flow section. A Level 2 decomposition should be read as a logical data-flow model rather than an implementation sequence: the browser submits an action through the API proxy, the backend validates identity/authorization, a service or route performs a database operation, and the resulting state is returned to the frontend.

14. Use Case Diagramm

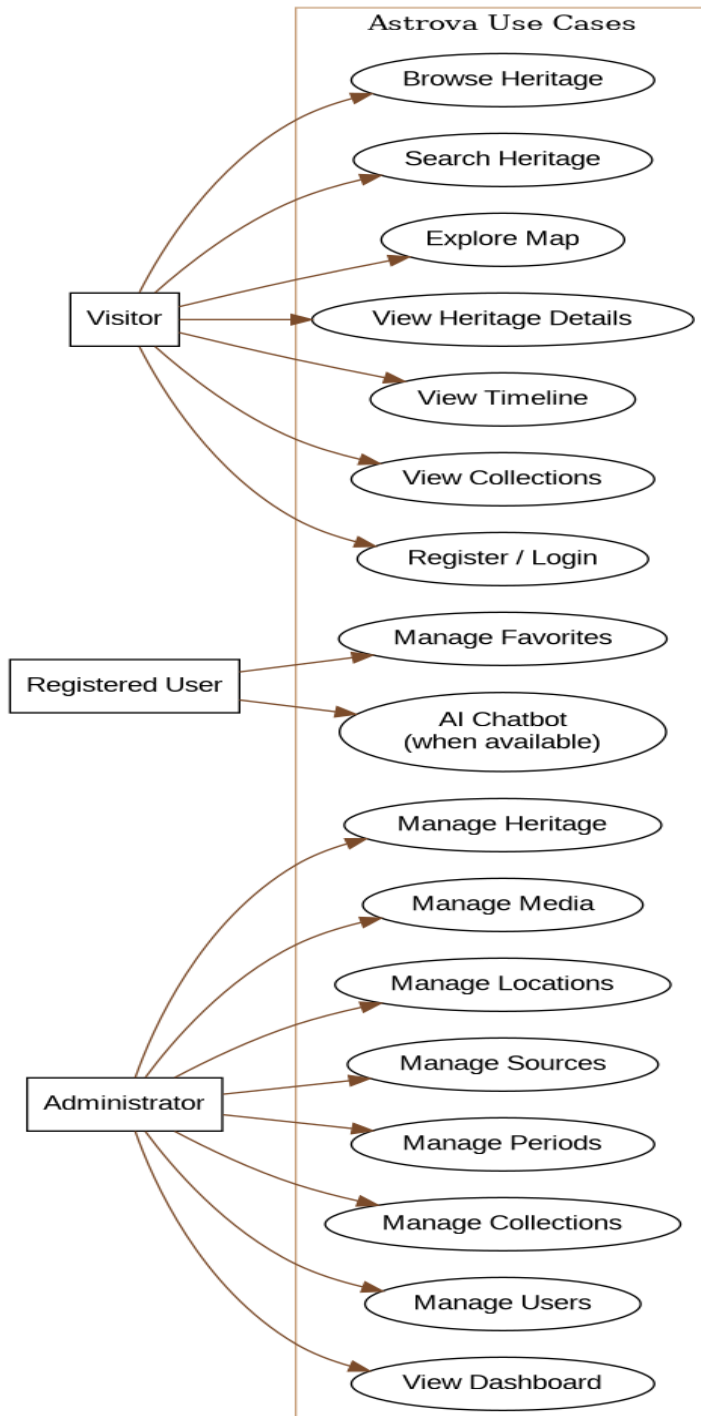


Figure 5. Use Case Diagram derived from the supplied actor/use-case definitions.

The three principal actors are Visitor, Registered User and Administrator. Registered users inherit the visitor discovery experience and gain personalized favorites. Administrators gain protected CRUD and dashboard capabilities.

15. Class / Component Architecture

Astrova is implemented as a TypeScript full-stack application rather than a traditional object-oriented Java/C++ system. Therefore, a useful class diagram should represent domain entities alongside route/service/middleware modules. The supplied project data identifies the following conceptual groups.

Layer	Representative modules/classes	Responsibility
Domain/Data	HeritageEntity, Location, HistoricalPeriod, Media, Source, Collection, User, UserFavorite	Represent persistent business entities.
Frontend	Auth hooks, Favorites hook, API client/proxy, feature components	Render UI and coordinate user interaction.
Backend Routes	Heritage, Search, Locations, Media, Sources, Periods, Collections, Favorites, Admin	Expose HTTP operations.
Services	Chatbot service and domain-oriented query logic	Business/data processing.
Middleware	requireAuth, optionalAuth, API-key validation, Admin token validation, rate limiting	Cross-cutting security and request controls.

Because the supplied package does not contain a rendered class diagram, this section intentionally describes the architecture at the module/domain level instead of inventing class methods that may not exist.

16. Activity Diagram

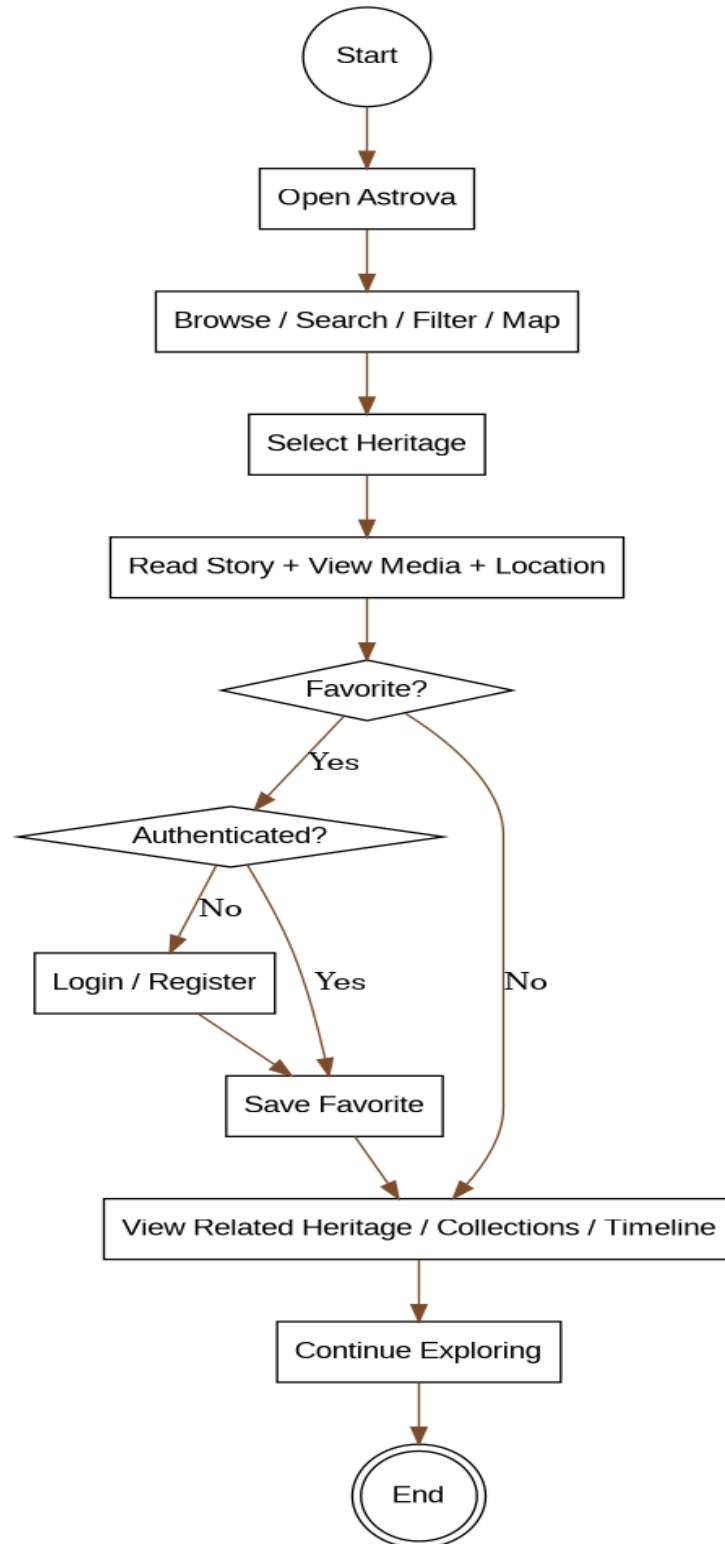


Figure 6. Primary heritage discovery and favorites activity flow.

The activity begins with discovery and ends with continued exploration. A favorite action requires authentication; unauthenticated users are routed through login/registration before the favorite can be persisted.

17. User Flow

1. Discover Astrova
2. Search, filter or explore the map
3. Select a heritage entity
4. Read the heritage story and inspect media/location
5. Explore related heritage, collections or timeline
6. Optionally favorite the heritage item
7. If not authenticated, sign in or create an account
8. Continue exploring

This flow reflects the supplied product journey and connects discovery, understanding, evidence/location, personalization and continued exploration.

18. API Architecture

The project uses a Next.js API proxy between browser-side code and the Express backend. This protects server-side API credentials and centralizes cookie forwarding. The supplied source-of-truth identifies public/data APIs, authentication APIs, favorites APIs and admin APIs.

18.1 API Categories

Category	Examples	Protection
Health	GET /api/health	None
Heritage & discovery	/api/heritage, /api/search, /api/locations, /api/media, /api/sources, /api/periods, /api/timeline	X-API-Key
Authentication	/api/auth/register, /api/auth/login, /api/auth/logout, /api/auth/me	Route-specific / JWT cookie
Favorites	/api/favorites/*	JWT cookie
Administration	/api/admin/*	X-Admin-Token

18.2 Complete API Inventory

Method	Endpoint	Purpose	Authentication
GET	/api/health	Health check	None
GET	/api/heritage	List heritage entities	API Key
GET	/api/heritage/:id	Get heritage by ID or slug	API Key
GET	/api/locations	List locations	API Key
GET	/api/media	List media	API Key
GET	/api/sources	List sources	API Key
GET	/api/periods	List historical periods	API Key
GET	/api/timeline	Get timeline data	API Key
GET	/api/collections	List collections	API Key
GET	/api/collections/:slug	Get collection by slug	API Key
GET	/api/search	Full-text search	API Key
GET	/api/search/suggestions	Search suggestions	API Key
GET	/api/states/counts	State heritage counts	API Key
GET	/api/ai/welcome	AI welcome message	API Key
POST	/api/auth/register	User registration	None (rate-limited)
POST	/api/auth/login	User login	None (rate-limited)
POST	/api/auth/logout	User logout	None
GET	/api/auth/me	Get current user	JWT Cookie
GET	/api/favorites	Get user favorites	JWT Cookie
POST	/api/favorites/:heritageId	Add favorite	JWT Cookie
DELETE	/api/favorites/:heritageId	Remove favorite	JWT Cookie
GET	/api/favorites/:heritageId/status	Check favorite status	JWT Cookie
POST	/api/favorites/sync	Sync favorites	JWT Cookie
GET	/api/admin/overview	Admin dashboard stats	X-Admin-Token
GET	/api/admin/heritage	Admin heritage list	X-Admin-Token
POST	/api/admin/heritage	Create heritage	X-Admin-Token
PUT	/api/admin/heritage/:id	Update heritage	X-Admin-Token
DELETE	/api/admin/heritage/:id	Delete heritage	X-Admin-Token
GET	/api/admin/media	Admin media list	X-Admin-Token
POST	/api/admin/media	Add media	X-Admin-Token
PUT	/api/admin/media/:id	Update media	X-Admin-Token
DELETE	/api/admin/media/:id	Delete media	X-Admin-Token
GET	/api/admin/locations	Admin locations list	X-Admin-Token
POST	/api/admin/locations	Create location	X-Admin-Token
PUT	/api/admin/locations/:id	Update location	X-Admin-Token
DELETE	/api/admin/locations/:id	Delete location	X-Admin-Token
GET	/api/admin/sources	Admin sources list	X-Admin-Token
POST	/api/admin/sources	Create source	X-Admin-Token
PUT	/api/admin/sources/:id	Update source	X-Admin-Token
DELETE	/api/admin/sources/:id	Delete source	X-Admin-Token

GET	/api/admin/users	Admin users list	X-Admin-Token
GET	/api/admin/users/:id	Get user detail	X-Admin-Token
DELETE	/api/admin/users/:id	Delete user	X-Admin-Token
GET	/api/admin/collections	Admin collections list	X-Admin-Token
POST	/api/admin/collections	Create collection	X-Admin-Token
PUT	/api/admin/collections/:id	Update collection	X-Admin-Token
DELETE	/api/admin/collections/:id	Delete collection	X-Admin-Token
POST	/api/admin/collections/:id/items	Add collection item	X-Admin-Token
DELETE	/api/admin/collections/:id/items/:itemId	Remove collection item	X-Admin-Token
GET	/api/admin/periods	Admin periods list	X-Admin-Token
POST	/api/admin/periods	Create period	X-Admin-Token
PUT	/api/admin/periods/:id	Update period	X-Admin-Token
DELETE	/api/admin/periods/:id	Delete period	X-Admin-Token

19. Authentication & Security

The supplied documentation describes JWT-based authentication using an HttpOnly `astrova_session` cookie. Passwords are hashed with `bcrypt`. Authentication middleware validates the JWT and the application uses route-specific protections. API keys are kept server-side through the Next.js proxy. Admin operations use a separate X-Admin-Token mechanism.

- JWT_SECRET is required; insecure fallback behavior was removed.
- API-key and Admin-token comparisons use timing-safe comparison.
- Authentication and favorite routes use route-level rate limits.
- Request body size is limited and CORS is configurable through allowed origins.
- UUID validation is applied where required by protected/admin operations.
- Production error responses avoid exposing stack traces.
- Password hashes are not exposed through user-facing/admin responses.

20. Admin Portal

The Admin Portal is a database-connected content management system protected by X-Admin-Token. The supplied project documentation identifies eight management areas plus a dynamic dashboard.

Admin Area	Capabilities
Dashboard	Database-backed statistics and operational overview.
Heritage	List, create, edit, delete and description management.
Media	Add/edit/delete media; image/video management; primary media integrity.
Locations	Create/edit/delete locations, coordinate validation and heritage counts.
Sources	Create/edit/delete reference sources.
Historical Periods	Create/edit/delete periods with BCE/CE-aware timeline data.

Collections	Create/edit/delete collections and manage collection items.
Users	List/view/delete users without exposing password hashes.

The supplied testing documentation reports missing Admin token → 401, incorrect token → 403 and correct token → 200 behavior. Admin end-to-end checks included creating and cleaning up test data and verifying public visibility.

21. Favorites & User Data Isolation

Favorites are designed as authenticated-user data. The backend requires authentication for listing, adding, deleting and status operations and derives the user identity from the authenticated session. The database enforces uniqueness on the user/heritage pair. The frontend uses shared authentication state and no longer treats anonymous localStorage favorites as an independent source of truth.

The supplied verification describes a multi-user isolation test: one user creates a favorite, a second user cannot see or delete the first user's favorite, the second user can create an independent favorite, and unauthenticated favorite API requests return 401. This is the intended security boundary for personalization.

22. Map & Heritage Discovery

The mapping subsystem uses Leaflet for interactive rendering and OpenStreetMap tiles. Location records contain latitude, longitude, type, state and district-related metadata. Heritage records can reference locations, allowing the detail experience and map to connect geographic context with heritage content.

The demo plan identifies the map as a key presentation screen: markers, popups, zoom controls and geographic discovery should be visible in the final demonstration.

23. Search & Filtering

Search is exposed through dedicated search and suggestion endpoints and is integrated with the heritage discovery interface. The source-of-truth identifies full-text search over heritage data and search suggestions. Filtering and discovery are also supported through map, state, category, period and collection-oriented experiences where implemented.

24. Collections

Collections provide curated groupings of heritage entities. The collections table stores collection metadata, while collection_items provides the many-to-many association and display ordering. The Admin Portal can create and manage collections and their items. The documented data set contains 6 collections and 98 collection items.

25. Timeline

The timeline organizes heritage records around 9 historical periods. Each period contains temporal information and can be associated with multiple heritage entities. The timeline supports chronological exploration and BCE/CE presentation. The backend was documented as using a consolidated query for timeline data rather than the earlier N+1 pattern.

26. AI / Chatbot Architecture

Astrova retains a multilingual chatbot architecture and 107 chatbot knowledge records, but the supplied project status explicitly marks the chatbot as under construction. The preserved data model includes chatbot_knowledge, conversations and conversation_messages. The design supports language, intent, question/answer and contextual heritage/location/period fields.

Because the chatbot is not currently a completed user-facing feature, this report does not present it as production-complete. Its presence in the database and architecture is documented as preparation for future multilingual AI assistance.

27. Testing & Verification

The supplied project materials describe comprehensive verification across APIs, authentication, favorites isolation, Admin CRUD, TypeScript and builds. This report distinguishes documented verification from assumptions and uses the latest supplied source-of-truth package.

Verification Area	Documented Result
API regression	Project package reports broad endpoint regression coverage; exact historical test counts are inconsistent across earlier reports.
Admin security	Missing token 401; wrong token 403; correct token 200 documented.
Favorites isolation	Multi-user isolation and anonymous rejection documented.
TypeScript	Frontend/backend type checking documented as passing.
Build	Frontend/backend builds documented as passing.
Database integrity	CRUD and public API persistence checks documented.

28. Bugs Found & Resolved

Issue	Root Cause / Symptom	Resolution
Location rendering	PostgreSQL decimal values could arrive as strings, causing `.toFixed()` runtime errors.	Normalize numeric values with Number() and finite-value checks before formatting.
Historical Period administration	Periods were previously read-only in Admin.	Added full CRUD and safe deletion behavior.
Media type mismatch	Admin media creation exposed unsupported types for the	New Admin media is restricted to Image/Video; existing historical

	intended workflow.	media types are preserved.
Heritage detail video presentation	Video presentation required professional playback behavior.	Heritage detail video uses muted, inline, loop-oriented playback.
Heritage detail location	Detail query could omit location data.	Added the required location join/fields.
Chatbot state overview	State overview data lacked heritage IDs/cards.	Selected IDs and returned usable heritage card information.
Vav intent recognition	Specific vav/ni vav/ki vav phrasing was not recognized.	Expanded intent/keyword handling for this heritage terminology.
Favorites authentication	Anonymous/shared localStorage behavior could mix user state.	Favorites now require authenticated user state and backend ownership enforcement.

29. Current Database Statistics

Data Area	Count
Heritage entities	74
Locations	54
Historical periods	9
Media	72
Sources	18
Heritage relationships	49
Collections	6
Collection items	98
Supported states	12
Chatbot knowledge	107
Users	2+ (documented)
Conversations	0
Conversation messages	0

Counts are reproduced from the supplied diagram-data package. They are documentation-package values dated September 3, 2026 and should be rechecked against the live database before final submission if the database has changed since that package was generated.

30. Limitations

- The chatbot remains under construction.
- No WebSocket-based real-time interaction is documented.
- No batch administrative operations are documented.
- Some list/query operations are limited rather than fully paginated.
- Media management is URL-based rather than a dedicated binary upload system.
- Relationship management does not have a full Admin UI.
- Audit logging is not documented as implemented.
- Some historical documentation contains conflicting endpoint/test-count and Git-status statements; the current source-of-truth should be treated as the basis for future cleanup.

31. Future Scope

- Complete and validate the multilingual AI chatbot.
- Add richer source citation and evidence presentation.
- Introduce pagination and more scalable data loading.
- Add batch Admin operations and audit logs.
- Improve media upload/verification workflows.
- Add richer relationship management.
- Enhance map clustering and heritage/list synchronization.
- Improve accessibility and image optimization.
- Expand supported states and heritage coverage.
- Consider PWA/offline capabilities for field and travel use cases.

32. Conclusion

Astrova demonstrates a substantial full-stack heritage discovery and content-management platform. Its strongest implemented areas are structured heritage data, geographic exploration, historical organization, collections, authentication, isolated favorites and a protected Admin Portal. The architecture is layered and database-driven, enabling the same heritage records to support multiple discovery experiences. The chatbot is intentionally preserved but remains under construction. The project therefore has a strong technical foundation for a Smart India Hackathon demonstration and for continued development into a broader multilingual heritage platform.

33. References

- Links :- 1. <https://www.indianculture.gov.in/>
- 2. <https://www.incredibleindia.gov.in>
- 3. <https://ich.unesco.org/en>
- 4. <https://nationalarchives.nic.in/>

Appendix A — Demonstration Screens

Screen	Route	Purpose
Homepage	/	Hero, navigation, featured heritage and state discovery.
Heritage Listing	/heritage	Heritage cards and discovery/filtering.
Heritage Detail	/heritage/[id]	Story, metadata, media, location, sources and related heritage.
Interactive Map	/map	Leaflet/OpenStreetMap map with heritage/location markers.
Search	/search	Search input, suggestions and results.
Timeline	/timeline	Historical periods and chronological heritage.

Collections	/collections	Curated collections.
Collection Detail	/collections/[slug]	Collection contents and relationships.
Login/Register	/auth	Authentication screens.
Favorites	/favorites	Authenticated personalized heritage list.
Admin Dashboard	/admin	Protected database-backed dashboard.
Admin Content Management	/admin	Heritage, media, locations, sources, periods, collections and users.